

# Och: A Core Calculus with Iota/Fix and Equirecursive Subtyping

Och is a dependently-typed  $\lambda$ -calculus with two recursive-binder forms —  $\iota$  (Cedille-style self-types) and **fix** (general recursion at the type and term level) — and a single syntactic category in which terms and types coincide. The checker is a structural type-synthesis walk layered on a substitution-based WHNF evaluator, and the subtyping relation is equirecursive with Brandt–Henglein coinductive hypotheses.

This document specifies Och as a type system: the term syntax, the concrete operational semantics, the declarative subtyping relation, and the algorithmic typing/subtyping judgments. Throughout, binders and variables are written with ordinary names ( $x, y, x, A, B, f, a, \dots$ ) and substitution is written  $\text{body}[x \mapsto v]$ . Binders are annotated with their bound name:  $\lambda(x \leq A).e$ ,  $\iota(x \leq A).B$ ,  $\text{fix}(x \leq A).b$ ,  $\text{let } x = v \text{ in body}$ . Context lookup is  $\Gamma(x)$ ; context extension is  $\Gamma, x \leq A$ .

## 1 Syntax

The term language is

|                                       |                                |
|---------------------------------------|--------------------------------|
| $e, \tau ::= x$                       | variable                       |
| $\top$                                | universe (top)                 |
| $\perp$                               | primitive bottom               |
| $\lambda(x \leq \tau).e$              | lambda                         |
| $e_1 e_2$                             | application                    |
| $\iota(x \leq \tau).e$                | self-type binder               |
| $\text{fix}(x \leq \tau).e$           | recursive binder               |
| $(e \leq \tau)$                       | ascription (erased at runtime) |
| $\text{let } x = e_1 \text{ in } e_2$ | let-binding                    |

There is no separate type category: every  $\tau$  above is itself a term. Throughout the paper  $\Gamma$  denotes a typing context (a finite map from variable names to their declared types).

**Variables, lambdas, application** are the standard  $\lambda$ -calculus.

$\top$  **and**  $\perp$  are the top and bottom of the subtyping lattice: every term subtypes  $\top$ , and  $\perp$  subtypes everything.

**Iota and fix** both bind a self-reference with an annotation  $\tau$  recording its type, but differ in how they unfold. A value  $v \leq \iota(x \leq A).B$  inhabits  $B[x \mapsto v]$  — the body’s self-reference points at the inhabitant itself, giving dependent elimination (Cedille-style self-types [1]). A value  $v \leq \text{fix}(x \leq A).B$  equirecursively equals its unfolding  $B[x \mapsto v]$  — general recursion at the type and term level.

**Ascription**  $(e \leq \tau)$  asserts that  $e$  subtypes  $\tau$  and instructs the type checker to forget information: the checker sees  $\tau$  rather than the precise type of  $e$ .

**Let**  $\text{let } x = e_1 \text{ in } e_2$  is sugar that allows defining a variable without first giving it a type — the checker infers the type of  $e_1$  and binds  $x$  to it.

## 2 Concrete semantics — the evaluation judgment

The concrete evaluator is a substitution-based call-by-value big-step interpreter on closed terms. Lambdas,  $\iota$ , and **fix** are values; applications in function position unroll the recursive binder by substituting its self-reference, then re-apply.

Write the judgment  $e \Downarrow v$  for “closed term  $e$  concretely evaluates to value  $v$ .” Values  $v ::= \lambda(x \leq \tau).e \mid \iota(x \leq \tau).e \mid \text{fix}(x \leq \tau).e \mid \top \mid x \vec{e}$  (neutral application spine with a stuck variable head).

$$\begin{array}{c}
\frac{}{e \Downarrow e} \text{E-VAL} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2[x \mapsto v_1] \Downarrow v}{(\text{let } x = e_1 \text{ in } e_2) \Downarrow v} \text{E-LET} \\
\\
\frac{f \Downarrow \iota(x \leq \tau).b \quad a \Downarrow v_a}{(b[x \mapsto \iota(x \leq \tau).b)] \ v_a \Downarrow v} \text{E-APP-I} \\
\\
\frac{f \Downarrow f' \quad a \Downarrow v_a}{f \ a \Downarrow f' \ v_a} \text{E-APP-NEUTRAL} \\
\\
\frac{e \Downarrow v}{(e \leq \tau) \Downarrow v} \text{E-ASC} \\
\\
\frac{f \Downarrow \lambda(x \leq \tau).b \quad a \Downarrow v_a}{b[x \mapsto v_a] \Downarrow v} \text{E-APP-B} \\
\\
\frac{f \Downarrow \text{fix}(x \leq \tau).b \quad a \Downarrow v_a}{(b[x \mapsto \text{fix}(x \leq \tau).b)] \ v_a \Downarrow v} \text{E-APP-FIX}
\end{array}$$

Where [E-Val] applies when  $e \in \{\top, \lambda., \iota., \text{fix } ..\}$ , and [E-App-Neutral] when the head is stuck.

Free variables are not values. Ascription is erased at evaluation ([E-Asc]); the declarative [S-Asc-L] rule (§4.4) provides the corresponding transparency.

**$\perp$  is a self-evaluating value.** Like  $\top$ ,  $\perp$  evaluates to itself via [E-Val] and has no application dispatch arm, so applying an argument to  $\perp$  is *stuck* (parallel to  $\top$ -application). This is intentional:  $\perp$  is a type, not a callable. The typing discipline must ensure well-typed programs never reach  $\perp \ a$  at runtime.

This is the **operational specification** of the language.

### 3 Overview of the algorithmic relations

Three judgments make up the algorithmic side of Och. This section introduces each one's semantic role and explains how they compose; the full rules appear in §§5–7.

#### 3.1 $S; \Gamma \vdash a \sqsubseteq_e b$ — structural subtype check (§6)

Decides whether well-typed WHNF term  $a$  is a subtype of well-typed WHNF term  $b$  under context  $\Gamma$ , given coinductive hypotheses  $S$ . Both inputs must already be well-typed and in WHNF — the caller ( $\rightsquigarrow$ ) is responsible for ensuring this.

The check is purely structural: it pattern-matches on the head constructors of  $a$  and  $b$  and recurses. Under binders it opens both sides with a fresh variable and extends  $\Gamma$  with the narrower of the two domains (always the RHS, since the contravariance check  $\text{dom}_B \sqsubseteq \text{dom}_A$  has already passed). The seen-set  $S$  provides coinductive closure for equirecursive types (§4.3 of the declarative subtyping). Internally,  $\sqsubseteq_e$  normalises intermediate terms (e.g. after substituting into an unfolded body) via a WHNF evaluator (§5) — this is safe because substituting a well-typed term into a well-typed body preserves well-typedness.

The key property:  $\sqsubseteq_e$  is **monotone in  $\Gamma$**  — narrowing any context entry preserves the judgment.

#### 3.2 $\Gamma \vdash e \rightsquigarrow v$ — type synthesis (§7)

Validates that  $e$  is well-typed under context  $\Gamma$  and returns its WHNF  $v$  as a type-witness (since in Och a value *is* its own most-precise type, by [S-Refl]). This is where type checking happens: at every application  $f \ a$ , the walk synthesises both sides, exposes a  $\Pi$  from the function, and asks  $\sqsubseteq_e$  whether the argument inhabits the domain. Binder annotations must subtype  $\top$ .

The semantic guarantee of  $\rightsquigarrow$  is that the term it accepts will remain well-typed under any narrowing of the context — that is, when abstract variables are replaced by narrower (more specific) values. This is the bridge between static checking and runtime execution: when we check a function body  $b$  under  $(\Gamma, x : A)$  with  $x$  abstract, the synthesis walk asks  $\sqsubseteq_e$  questions like “does this expression inhabit the domain  $A$ ?” Later, when the evaluator runs  $b[x \mapsto v]$  with a concrete  $v$  that is narrower than  $A$ , every subtype question the walk asked is at least as easy — a narrower context can only make more things provable, never fewer. So the abstract check is a sound over-approximation of every possible concrete execution.

### 3.3 $e \Downarrow v$ — concrete evaluation (§2)

The reference big-step interpreter on closed terms (no open context). This is what actually runs programs. It is defined independently of the type-checker and has no access to a context or type information — it is a pure untyped evaluator.

The connection to the type-checker is through two soundness theorems: **preservation** ( $e \sqsubseteq \tau$  and  $e \Downarrow v$  imply  $v \sqsubseteq \tau$ ) and **progress** ( $\rightsquigarrow$  accepted  $e$  implies  $\Downarrow$  never gets stuck). Together these say: if you type-check first, the evaluator either produces a well-typed value or runs out of fuel — it never reaches a stuck state.

### 3.4 How they compose

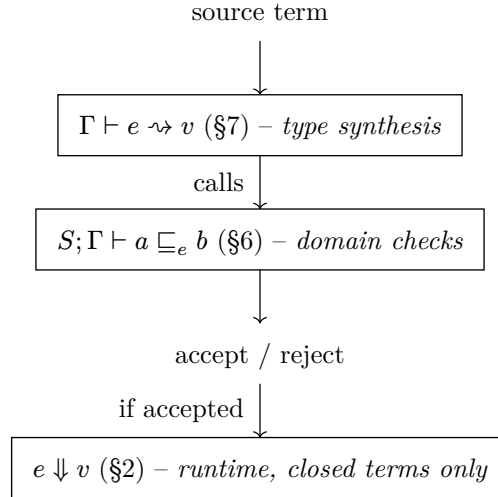


Figure 1: Static and runtime phases

The top box is the static phase:  $\rightsquigarrow$  walks the term, calling  $\sqsubseteq_e$  for each typing obligation. If the walk accepts, the term enters the runtime phase:  $\Downarrow$  evaluates it without any type information. Soundness (§8) connects the two phases — the static acceptance guarantees that the runtime evaluation is well-behaved.

## 4 Declarative subtyping

Subtyping in Och means set inclusion:  $A \sqsubseteq B$  iff every value of  $A$  is also a value of  $B$ . The relation is written

$$S; \Gamma \vdash a \sqsubseteq b$$

### 4.1 Context $\Gamma$

$\Gamma$  is the typing context — a finite ordered list of (variable, type) pairs, with the most recent binder at the front. Lookup is  $\Gamma(x)$  for the declared type of  $x$ ; extension is  $\Gamma, x \leq A$ .

### 4.2 Seen set $S$

$S$  is a set of pairs  $(a, b)$  — ancestor subtyping goals introduced by productive unfolding rules ( $\iota$ -introduction and the four unfold rules, §4.3). The invariant: only those five rules extend  $S$ ; every other rule propagates it unchanged. This is the Brandt–Henglein device [2] for equirecursive subtyping — coinductive assumptions are only legal after at least one productive step, so reflexivity of a non-productive goal cannot be closed by a hypothesis. In effect,  $S$  is a finite representation of a coinductive (potentially infinite) proof tree: when a goal recurs, the branch closes rather than recursing forever, encoding a regular infinite derivation as a finite one.

The “real” subtyping judgment is  $\emptyset; \Gamma \vdash a \sqsubseteq b$  (empty hypothesis set); non-empty  $S$  arises only inside a derivation.

### 4.3 Rule taxonomy

The rules fall into four categories. Each serves a distinct purpose; knowing which category a rule belongs to predicts its shape.

| Category                    | Purpose                                                                                   | Rules                                                   | Extends $S$ ? |
|-----------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------|---------------|
| Structural (§4.1)           | Plumbing: compose, lookup, without inspecting constructors on either side                 | S-Refl, S-Top, S-BotL, S-Trans, S-Hyp, S-Var            | no            |
| Congruence (§4.2)           | Match constructor on both sides; reduce to sub-obligations with variance                  | S-Lam, S-App-Cong, S-Iota-Cong, S-Fix-Cong, S-LetE-Cong | no            |
| Productive unfolding (§4.3) | Unfold a recursive binder ( $\iota/\text{fix}$ ); extend $S$ ; enable coinductive closure | S-Iota-Intro, S-Unfold-Iota-L/R, S-Unfold-Fix-L/R       | yes           |
| Conversion (§4.4)           | Close under head reduction so the algorithmic evaluator is sound                          | S-Beta-L/R, S-Let-L/R, S-Asc-L, S-Asc-L-Ann, S-Asc-R    | no            |

Table 1: Rule taxonomy for declarative subtyping

The  $S$ -extension column matters: S-Hyp can only fire against entries that some ancestor productive rule installed, so any path to S-Hyp must cross an unfold — the productivity requirement made mechanical.

### 4.4 Structural rules

“Structural” rules talk about  $\sqsubseteq$  as a relation on terms without inspecting either side’s head constructor: reflexivity, transitivity, the top element, hypothesis lookup, and variable lookup.

$$\begin{array}{ccc}
\frac{}{S; \Gamma \vdash e \sqsubseteq e} \text{S-REFL} & \frac{}{S; \Gamma \vdash e \sqsubseteq \top} \text{S-TOP} & \frac{}{S; \Gamma \vdash \perp \sqsubseteq e} \text{S-BOTL} \\
\frac{S; \Gamma \vdash a \sqsubseteq b \quad S; \Gamma \vdash b \sqsubseteq c}{S; \Gamma \vdash a \sqsubseteq c} \text{S-TRANS} & \frac{(a, b) \in S}{S; \Gamma \vdash a \sqsubseteq b} \text{S-HYP} & \frac{}{S; \Gamma \vdash x \sqsubseteq \Gamma(x)} \text{S-VAR}
\end{array}$$

**Ex falso via subsumption.** There is no dedicated “absurd” eliminator. If  $a \sqsubseteq \perp$  is derivable, then  $a \sqsubseteq e$  for every  $e$  via [S-Trans] on [S-BotL]. The “contradiction” discharge is subsumption alone. This matches the DOT tradition:  $\perp$  inhabits every type trivially in subtyping, so any term whose type is already  $\perp$  flows into any expected type without further ceremony.

**Why [S-Trans] is a constructor, not a derived theorem.** In a normal simply-typed subtyping relation, transitivity is admissible: a standard induction on the shape of the two derivations composes them. That argument requires a decreasing syntactic measure — typically, both derivations get strictly smaller in each case of the composition. Och’s four unfold rules and iota-intro break this: unfolding  $\text{fix}(x \leq A). \text{body}$  replaces it with  $\text{body}[x \mapsto \text{fix}(x \leq A). \text{body}]$ , which is *larger* than the original. There’s no obvious structural measure on derivations that decreases through an unfold, so transitivity is not derivable by induction on derivations. The seen-set discipline of §4.3 is the coinductive counterpart that keeps the relation consistent, but it doesn’t by itself give admissibility of transitivity.

Eliminating transitivity as a primitive rule (*transitivity elimination*) is highly desirable: it simplifies metatheory and makes the relation syntax-directed. Hutchins’ Pure Subtype Systems [3] — a closely related system — did not manage to eliminate transitivity. Pasquale and García-Pérez [4] succeeded in a continuation of that work, but required switching the entire theory to a Krivine machine.

### 4.5 Congruence rules

Congruence rules *do* inspect the head constructor on both sides, require it to match, and reduce the goal to sub-obligations on the immediate sub-terms with appropriate variance.

$$\begin{array}{c}
\frac{S; \Gamma \vdash A_2 \sqsubseteq A_1 \quad S; \Gamma, x \leq A_2 \vdash b_1 \sqsubseteq b_2}{S; \Gamma \vdash \lambda(x \leq A_1).b_1 \sqsubseteq \lambda(x \leq A_2).b_2} \text{S-LAM} \qquad \frac{S; \Gamma \vdash f_2 \sqsubseteq f_1 \quad S; \Gamma \vdash a_2 \sqsubseteq a_1}{S; \Gamma \vdash a_1 \sqsubseteq a_2} \text{S-APP-CONG} \\
\frac{S; \Gamma \vdash A_1 \sqsubseteq A_2 \quad S; \Gamma, x \leq A_2 \vdash B_1 \sqsubseteq B_2}{S; \Gamma \vdash \iota(x \leq A_1).B_1 \sqsubseteq \iota(x \leq A_2).B_2} \text{S-IOTA-CONG} \\
\frac{S; \Gamma \vdash A_1 \sqsubseteq A_2 \quad S; \Gamma, x \leq A_2 \vdash b_1 \sqsubseteq b_2}{S; \Gamma \vdash \text{fix}(x \leq A_1).b_1 \sqsubseteq \text{fix}(x \leq A_2).b_2} \text{S-FIX-CONG} \qquad \frac{S; \Gamma \vdash v_1 \sqsubseteq v_2 \quad S; \Gamma, x \leq v_2 \vdash b_1 \sqsubseteq b_2}{S; \Gamma \vdash \text{let } x = v_1 \text{ in } b_1 \sqsubseteq \text{let } x = v_2 \text{ in } b_2} \text{S-LETE-CONG}
\end{array}$$

The equivalence premise on [S-App-Cong] (both directions on the argument) is necessary because a neutral head can use its argument at any variance, so equivalence is the only sound congruence.

## 4.6 Productive unfolding (extend $S$ )

A rule is *productive* when it replaces a goal whose head is a recursive binder ( $\iota$  or  $\text{fix}$ ) with a goal where that binder has been unfolded once. Unfolding makes the term *larger* in syntactic size, so no structural induction can traverse an arbitrary chain of unfolds. Productivity instead provides a *coinductive* handle: once at least one unfold has fired, the original goal is guaranteed to eventually re-appear as an ancestor, at which point [S-Hyp] can close the derivation.

The rules below are the only ones that extend  $S$ . Let  $S' = S \cup \{(a, b)\}$  where  $(a, b)$  is the current goal.

$$\begin{array}{c}
\frac{S'; \Gamma \vdash a \sqsubseteq A \quad S'; \Gamma \vdash a \sqsubseteq B[x \mapsto a]}{S; \Gamma \vdash a \sqsubseteq \iota(x \leq A).B} \text{S-IOTA-INTRO} \qquad \frac{S'; \Gamma \vdash B[x \mapsto \iota(x \leq A).B] \sqsubseteq c}{S; \Gamma \vdash \iota(x \leq A).B \sqsubseteq c} \text{S-UNFOLD-IOTA-L} \\
\frac{S'; \Gamma \vdash a \sqsubseteq B[x \mapsto \iota(x \leq A).B]}{S; \Gamma \vdash a \sqsubseteq \iota(x \leq A).B} \text{S-UNFOLD-IOTA-R} \\
\frac{S'; \Gamma \vdash b[x \mapsto \text{fix}(x \leq A).b] \sqsubseteq c}{S; \Gamma \vdash \text{fix}(x \leq A).b \sqsubseteq c} \text{S-UNFOLD-FIX-L} \qquad \frac{S'; \Gamma \vdash a \sqsubseteq b[x \mapsto \text{fix}(x \leq A).b]}{S; \Gamma \vdash a \sqsubseteq \text{fix}(x \leq A).b} \text{S-UNFOLD-FIX-R}
\end{array}$$

S-Unfold-Iota-R is the weaker sibling of S-Iota-Intro (same conclusion, no annotation premise), needed to close the equivalence of  $\iota$ -unfolding.

**Worked example:  $\text{dtrue} \sqsubseteq \text{dBool}$ .** With

$$\begin{aligned}
\text{dtrue} &:= \iota(x \leq \top). \lambda(P \leq x \rightarrow \top). \lambda(t \leq P \ x). \lambda(f \leq \top). t \\
\text{dBool} &:= \text{fix}(B \leq \top). \iota(x \leq B). \lambda(P \leq B \rightarrow \top). \lambda(t \leq P \ \text{dtrue}). \lambda(f \leq P \ \text{dfalse}). P \ x
\end{aligned}$$

the subtyping check  $\emptyset; \Gamma \vdash \text{dtrue} \sqsubseteq \text{dBool}$  loops through a contravariant domain:  $\text{dBool}$ 's motive parameter is  $P \leq \text{dBool} \rightarrow \top$ , while  $\text{dtrue}$ 's is  $P \leq x \rightarrow \top$  where  $x$  is the inhabitant — ultimately  $\text{dtrue}$ . So the [S-Lam] contravariant premise for the  $P$  binder demands  $\text{dBool} \sqsubseteq \text{dtrue}$ ... which needs the original relationship back. Without the seen set this loops forever; with it, the productive unfold at the top installs  $(\text{dtrue}, \text{dBool})$  into  $S$ , and the loop closes via [S-Hyp]:

$$\begin{array}{l}
\emptyset; \Gamma \vdash \text{dtrue} \sqsubseteq \text{dBool} \qquad \text{root goal} \\
S_1; \Gamma \vdash \text{dtrue} \sqsubseteq \iota(x \leq \text{dBool}). \lambda(P \leq \text{dBool} \rightarrow \top). \lambda(t \leq P \ \text{dtrue}). \lambda(f \leq P \ \text{dfalse}). P \ x \quad \text{S-Unfold-Fix-R} \\
\text{where } S_1 = \{(\text{dtrue}, \text{dBool})\} \\
\text{Annotation premise:} \\
S_1; \Gamma \vdash \text{dtrue} \sqsubseteq \text{dBool} \qquad \text{S-Hyp } \checkmark \\
\text{root goal reappears — closed by } (\text{dtrue}, \text{dBool}) \in S_1 \\
\text{Body premise (after } [x \mapsto \text{dtrue}]): \\
S_1; \Gamma \vdash \text{dtrue} \sqsubseteq \lambda(P \leq \text{dBool} \rightarrow \top). \lambda(t \leq P \ \text{dtrue}). \lambda(f \leq P \ \text{dfalse}). P \ \text{dtrue} \quad \text{S-Lam } \dots \\
\text{further structural reduction; any recurrence of } \text{dtrue} \sqsubseteq \text{dBool} \text{ closes via S-Hyp}
\end{array}$$

Both branches of [S-Iota-Intro] find the root goal waiting in  $S_1$  and close via [S-Hyp]. This is the Brandt–Henglein discipline in action: recursion in the subtyping judgment is legal *only across a productive unfold*, so non-productive loops (e.g. reflexivity-by-loop) cannot sneak through.

## 4.7 Conversion

The algorithm normalises before comparing, so the declarative relation must be closed under head reduction. These rules provide that closure.

$$\begin{array}{c}
\frac{S; \Gamma \vdash \text{body}[x \mapsto \arg] \sqsubseteq b}{S; \Gamma \vdash (\lambda(x \leq A). \text{body}) \arg \sqsubseteq b} \text{S-BETA-L} \qquad \frac{S; \Gamma \vdash a \sqsubseteq \text{body}[x \mapsto \arg]}{S; \Gamma \vdash a \sqsubseteq (\lambda(x \leq A). \text{body}) \arg} \text{S-BETA-R} \\
\\
\frac{S; \Gamma \vdash \text{body}[x \mapsto \text{val}] \sqsubseteq b}{S; \Gamma \vdash \text{let } x = \text{val in body} \sqsubseteq b} \text{S-LET-L} \qquad \frac{S; \Gamma \vdash a \sqsubseteq \text{body}[x \mapsto \text{val}]}{S; \Gamma \vdash a \sqsubseteq \text{let } x = \text{val in body}} \text{S-LET-R} \\
\\
\frac{S; \Gamma \vdash e \sqsubseteq b}{S; \Gamma \vdash (e \leq \tau) \sqsubseteq b} \text{S-ASC-L} \qquad \frac{S; \Gamma \vdash \tau \sqsubseteq b}{S; \Gamma \vdash (e \leq \tau) \sqsubseteq b} \text{S-ASC-L-ANN} \qquad \frac{S; \Gamma \vdash a \sqsubseteq e}{S; \Gamma \vdash a \sqsubseteq (e \leq \tau)} \text{S-ASC-R}
\end{array}$$

**Ascription rules: mixed transparency.** Ascription has three rules, not two. On the **RHS**, [S-Asc-R] sees through to the inner value  $e$  — the ascription doesn't widen the set of values it contains. On the **LHS**, both rules coexist: [S-Asc-L] (transparent, sees  $e$ ) is needed because the runtime erases ascriptions, while [S-Asc-L-Ann] (narrowing, sees  $\tau$ ) is what the algorithmic checker uses — it peels an LHS ascription to the annotation, so the caller sees  $\tau$ , not the precise inner value.

Given  $A \sqsubseteq B \sqsubseteq C$ , the mixed rules give the expected behavior:

| Goal                       | Rule      | Reduces to        | Holds? |
|----------------------------|-----------|-------------------|--------|
| $(A \leq B) \sqsubseteq C$ | Asc-L-Ann | $B \sqsubseteq C$ | yes    |
| $A \sqsubseteq (B \leq C)$ | Asc-R     | $A \sqsubseteq B$ | yes    |
| $B \sqsubseteq (A \leq C)$ | Asc-R     | $B \sqsubseteq A$ | no     |
| $(A \leq C) \sqsubseteq B$ | Asc-L-Ann | $C \sqsubseteq B$ | no     |

## 5 Substitution-based evaluation

The WHNF evaluator takes an expression and reduces it to weak head normal form via syntactic substitution. Input and output are both terms — there is no separate value type, no closures, no environments, and no quoting step.

### 5.1 Design

The key simplification: because the term language serves as both AST and value,  $\beta$ -reduction is literal syntactic substitution —  $\text{body}[x \mapsto \arg]$ . Lambdas,  $\iota$ -binders, and fix-binders are already in WHNF and evaluate to themselves. Applications reduce by substituting the evaluated argument into the function's body.

Two fuel parameters bound the computation:

- **fuel**: overall step budget. Every recursive call decrements it.
- **unf**: unfold budget for  $\iota$ /fix heads in application position. Decrement on each self-referential unfold; when exhausted, the application is left as a stuck neutral ( $f \ a$ ). Prevents divergence on non-terminating self-application chains.

The lex ordering (fuel, unf) gives a structurally decreasing termination measure.

### 5.2 Level-variables for open terms

The subtype checker (§6) needs to compare terms under binders — e.g. to check  $\lambda(x \leq A).b_1 \sqsubseteq \lambda(x \leq A).b_2$ , it opens both bodies with a fresh variable and recurses. Rather than introduce a separate value domain with neutral placeholders, the engine encodes free variables as **level-variables**: indices in a reserved high range. A level-variable represents the free variable introduced at a given *level* (counted from the context root, not the innermost binder). Level-vars are stable under further binder-openings — opening a new binder introduces a new level-var without shifting existing ones.

This encoding lets the entire algorithmic pipeline — evaluation, subtype checking, type synthesis — operate on a single term type with no separate value/neutral/closure machinery.

### 5.3 Evaluation rules

Write the judgment  $e \Rightarrow v$  for “the evaluator reduces  $e$  to WHNF  $v$ .”

$$\begin{array}{c}
\frac{}{\top \Rightarrow \top} \text{ES-TYPE} \qquad \frac{}{\perp \Rightarrow \perp} \text{ES-BOT} \qquad \frac{}{\lambda(x \leq A).b \Rightarrow \lambda(x \leq A).b} \text{ES-LAM} \\
\frac{}{\iota(x \leq A).b \Rightarrow \iota(x \leq A).b} \text{ES-IOTA} \qquad \frac{}{\text{fix}(x \leq A).b \Rightarrow \text{fix}(x \leq A).b} \text{ES-FIX} \qquad \frac{}{x \Rightarrow x} \text{ES-LEVELVAR} \\
\frac{e \Rightarrow e' \quad \tau \Rightarrow \tau'}{(e \leq \tau) \Rightarrow (e' \leq \tau')} \text{ES-ASC} \qquad \frac{\text{val} \Rightarrow v_1 \quad \text{body}[x \mapsto v_1] \Rightarrow v}{\text{let } x = \text{val} \text{ in body} \Rightarrow v} \text{ES-LET} \\
\frac{f \Rightarrow \lambda(x \leq A). \text{body} \quad a \Rightarrow v_a \quad \text{body}[x \mapsto v_a] \Rightarrow v}{f a \Rightarrow v} \text{ES-APP-B} \\
\frac{f \Rightarrow \iota(x \leq A). \text{body} \quad a \Rightarrow v_a \quad (\text{body}[x \mapsto f]) v_a \Rightarrow v}{f a \Rightarrow v} \text{ES-APP-I-UNFOLD} \qquad \frac{f \Rightarrow \text{fix}(x \leq A). \text{body} \quad a \Rightarrow v_a \quad (\text{body}[x \mapsto f]) v_a \Rightarrow v}{f a \Rightarrow v} \text{ES-APP-FIX-UNFOLD}
\end{array}$$

Where  $\iota/\text{fix}$  unfold rules require  $\neg \text{neutral}(v_a) \wedge \text{unf} > 0$ . When  $\text{neutral}(v_a) \vee \text{unf} = 0$ , the application is stuck:

$$\begin{array}{c}
\frac{f \Rightarrow f' \quad a \Rightarrow v_a}{f a \Rightarrow f' v_a} \text{ES-APP-I-STUCK} \qquad \frac{f \Rightarrow f' \quad a \Rightarrow v_a}{f a \Rightarrow f' v_a} \text{ES-APP-FIX-STUCK} \\
\frac{f \Rightarrow (\text{inner} \leq \tau) \quad \text{inner} \quad a \Rightarrow v}{f a \Rightarrow v} \text{ES-APP-ASC} \qquad \frac{f \Rightarrow f' \quad a \Rightarrow v_a}{f a \Rightarrow f' v_a} \text{ES-APP-NEUTRAL}
\end{array}$$

Where [ES-App-Neutral] applies when  $\text{neutral}(f') \vee f' \in \{\top, \perp\}$ .

## 6 Algorithmic subtyping on expressions

The algorithmic subtype checker is the realisation of §4. It operates directly on terms in WHNF. The subtype checker and the WHNF evaluator form a mutual block with a *lex (fuel, phase)* termination measure — the checker calls the evaluator to force WHNF before comparing, and calls itself recursively under binders.

Write the judgment  $S; \Gamma \vdash a \sqsubseteq_e b$  for the algorithmic check, where  $a, b$  are in WHNF,  $S$  is a list of pairs (the seen-set), and  $\Gamma$  maps level indices to their declared types.

### 6.1 Top-level dispatch

Before case-splitting on shapes, the checker forces both sides to WHNF, then **peels ascriptions asymmetrically**:

- LHS ascription  $\rightarrow$  replace with the annotation (narrowing)
- RHS ascription  $\rightarrow$  replace with the inner value (content)

After peeling, three fast paths fire in order:

1. **Syntactic equality** ( $a'' = b''$ ): immediate accept.
2. **Seen-set hit** ( $(a'', b'') \in S$ ): coinductive accept.
3. **Top** ( $b'' = \top$ ): immediate accept.

If none fires, it delegates to the per-shape case analysis.

## 6.2 Per-shape rules

$$\begin{array}{c}
\frac{}{S; \Gamma \vdash \perp \sqsubseteq_e b} \text{AS-BOTL} \\
\frac{S; \Gamma \vdash A_2 \sqsubseteq_e A_1}{S; \Gamma, x \leq A_2 \vdash b_1[\text{fresh}] \sqsubseteq_e b_2[\text{fresh}]} \text{AS-LAM} \\
\frac{S; \Gamma \vdash A_1 \sqsubseteq_e A_2 \quad S; \Gamma, x \leq A_2 \vdash B_1[\text{fresh}] \sqsubseteq_e B_2[\text{fresh}]}{S; \Gamma \vdash \iota(x \leq A_1).B_1 \sqsubseteq_e \iota(x \leq A_2).B_2} \text{AS-IOTA-IOTA}
\end{array}$$

With fallback to iota-intro if structural match fails. Similarly:

$$\frac{S; \Gamma \vdash A_1 \sqsubseteq_e A_2 \quad S; \Gamma, x \leq A_2 \vdash b_1[\text{fresh}] \sqsubseteq_e b_2[\text{fresh}]}{S; \Gamma \vdash \text{fix}(x \leq A_1).b_1 \sqsubseteq_e \text{fix}(x \leq A_2).b_2} \text{AS-FIX-FIX}$$

With fallback to unfold-R if structural match fails.

$$\begin{array}{c}
\frac{S'; \Gamma \vdash a \sqsubseteq_e A}{S'; \Gamma \vdash a \sqsubseteq_e B[x \mapsto a]} \text{AS-IOTA-INTRO} \\
\frac{S'; \Gamma \vdash a \sqsubseteq_e b[x \mapsto \text{fix}(x \leq A).b]}{S; \Gamma \vdash a \sqsubseteq_e \text{fix}(x \leq A).b} \text{AS-UNFOLD-FIX-R} \\
\frac{S'; \Gamma \vdash b[x \mapsto \text{fix}(x \leq A).b] \sqsubseteq_e c}{S; \Gamma \vdash \text{fix}(x \leq A).b \sqsubseteq_e c} \text{AS-UNFOLD-FIX-L} \quad \frac{S'; \Gamma \vdash B[x \mapsto \iota(x \leq A).B] \sqsubseteq_e c}{S; \Gamma \vdash \iota(x \leq A).B \sqsubseteq_e c} \text{AS-UNFOLD-IOTA-L}
\end{array}$$

Where  $S' = (a, b) :: S$  in each productive rule.

## 6.3 Neutral handling

When both sides are neutral (level-var-headed spines), the checker tries **spine comparison**: heads must be the same level-var; arguments must be pairwise *equivalent* (checked in both directions, matching [S-App-Cong]).

If spine comparison fails, or if only the LHS is neutral, the checker falls back to **neutral ascent**: it synthesises the LHS neutral's type by walking its spine through  $\Gamma$ , then recurses with the synthesised type on the LHS. This corresponds to [S-Var] in the declarative relation — a neutral ascends to its declared type.

$$\begin{array}{c}
\frac{S; \Gamma \vdash e_i \sqsubseteq_e e'_i \quad S; \Gamma \vdash e'_i \sqsubseteq_e e_i}{S; \Gamma \vdash x e_1 \dots e_n \sqsubseteq_e x e'_1 \dots e'_n} \text{AS-SPINE} \\
\frac{\text{synthType}(\Gamma, n) = \text{some } \tau}{S; \Gamma \vdash \tau \sqsubseteq_e b} \text{AS-ASCENT} \\
S; \Gamma \vdash n \sqsubseteq_e b
\end{array}$$

## 6.4 $\Pi$ -exposure helper

The  $\Pi$ -exposure helper unfolds a  $\text{fix}/\iota$  wrapper in a type until a  $\lambda$  ( $\Pi$ ) head is exposed. For  $\text{fix}$ , the self-reference is the  $\text{fix}$  itself ( $\mu$ -unfold); for  $\iota$ , the self-reference is the *inhabitant* whose type is being computed —  $n \leq \iota(x \leq A).B$  means  $n \leq B[x \mapsto n]$ . Returns nothing on  $\perp$  (Bot is not a  $\Pi$  head).

## 7 Algorithmic typing (synthesis)

The type-synthesis walk is a single structural recursion over terms that delegates all typing questions to the subtype checker (§6). Where a bidirectional checker dispatches between synthesis and checking modes, this walk has no separate checking mode — it asks the *complete* structural engine to decide each domain check directly.

### 7.1 Design

The synthesis walk over  $\Gamma$  and  $e$  returns the WHNF of  $e$  — which, by Och's Refl-typing convention ( $a \sqsubseteq a$ ), doubles as the most-precise type. At every node that introduces a typing obligation (applications, ascriptions), it calls the subtype checker to verify the obligation. The  $\Gamma$  argument maps level indices to the WHNF types of free variables introduced by binder-opening.

### 7.2 Synthesis rules

$$\begin{array}{c}
\frac{}{\Gamma \vdash \top \rightsquigarrow \top} \text{I-TYPE} \quad \frac{}{\Gamma \vdash \perp \rightsquigarrow \perp} \text{I-BOT} \quad \frac{}{\Gamma \vdash x \rightsquigarrow x} \text{I-VAR}
\end{array}$$



$$\begin{array}{c}
\frac{\emptyset; \Gamma \vdash A \sqsubseteq_e \top \quad \Gamma, x \leq \text{whnf}(A) \vdash b[\text{fresh}] \rightsquigarrow \cdot}{\Gamma \vdash \lambda(x \leq A).b \rightsquigarrow \text{whnf}(\lambda(x \leq A).b)} \text{I-LAM} \\
\\
\frac{\emptyset; \Gamma \vdash A \sqsubseteq_e \top \quad \Gamma, x \leq \text{whnf}(A) \vdash b[\text{fresh}] \rightsquigarrow \cdot}{\Gamma \vdash \iota(x \leq A).b \rightsquigarrow \text{whnf}(\iota(x \leq A).b)} \text{I-IOTA} \quad \frac{\emptyset; \Gamma \vdash A \sqsubseteq_e \top \quad \Gamma, x \leq \text{whnf}(A) \vdash b[\text{fresh}] \rightsquigarrow \cdot}{\Gamma \vdash \text{fix}(x \leq A).b \rightsquigarrow \text{whnf}(\text{fix}(x \leq A).b)} \text{I-FIX} \\
\\
\frac{\emptyset; \Gamma \vdash \tau \sqsubseteq_e \top \quad \Gamma \vdash e \rightsquigarrow v \quad \emptyset; \Gamma \vdash v \sqsubseteq_e \text{whnf}(\tau)}{\Gamma \vdash (e \leq \tau) \rightsquigarrow \text{whnf}(e \leq \tau)} \text{I-ASC} \\
\\
\frac{\Gamma \vdash \text{val} \rightsquigarrow v_{\text{val}} \quad \Gamma, x \leq v_{\text{val}} \vdash \text{body}[\text{fresh}] \rightsquigarrow \cdot}{\Gamma \vdash \text{let } x = \text{val in body} \rightsquigarrow \text{whnf}(\text{let } x = \text{val in body})} \text{I-LET} \\
\\
\frac{\Gamma \vdash f \rightsquigarrow v_F \quad \Gamma \vdash a \rightsquigarrow v_A \quad \pi_{\text{expr}} \leftarrow \text{exposePi}(v_F) \quad \emptyset; \Gamma \vdash v_A \sqsubseteq_e \text{dom}}{\Gamma \vdash f \ a \rightsquigarrow \text{whnf}(\pi_{\text{expr}} \ a_v)} \text{I-APP}
\end{array}$$

**Refl-typing convention.** Canonical forms (lam, iota, fix) synthesise themselves: the walk returns  $\text{whnf}(e)$  for these nodes. Since  $a \sqsubseteq a$  holds declaratively by [S-Refl], the synthesised WHNF is trivially a valid type-witness.

**$\perp$  in synthesis.**  $\perp$  synthesises itself via [I-Bot].  $\perp$  is not a  $\Pi$  head, so applying an argument to a  $\perp$ -typed function fails at the  $\Pi$ -exposure step in [I-App].

### 7.3 Abstract interpretation of bodies

When the synthesis walk enters a binder (lam, iota, fix, let), it opens the body by substituting a fresh level-variable for the bound variable and extends  $\Gamma$  with the annotation's WHNF. The level-variable is **neutral** with respect to evaluation — it evaluates to itself, blocks  $\beta$ -reduction, and propagates through application spines as a stuck head. This is the substitution-based analogue of opening a closure with a fresh free variable: the level-var acts as a symbolic “any value of this type,” and the structural subtype checker can compare bodies containing level-vars by spine-walking and ascending through  $\Gamma$ .

## 8 Metatheory

Soundness connects the concrete semantics (§2), the declarative subtyping (§4), and the algorithmic checker (§§5–7).

**Subtyping soundness.** If the algorithm accepts two values, they are declaratively subtypes.

$$\text{subCheck } a \ b = \text{ok} \Rightarrow \emptyset; \emptyset \vdash a \sqsubseteq b$$

**Synthesis soundness.** If synthesis accepts  $e$ , then  $e$  declaratively subtypes its synthesised WHNF.

$$\text{synth } e = \text{ok } v \Rightarrow \emptyset; \emptyset \vdash e \sqsubseteq v$$

**Evaluation equivalence.** If  $e$  evaluates to  $e'$ , both directions of subtyping hold (**sorry-free**).

$$e \text{ closed} \wedge e \Downarrow e' \Rightarrow \emptyset; \emptyset \vdash e' \sqsubseteq e \wedge \emptyset; \emptyset \vdash e \sqsubseteq e'$$

**Preservation.** Standard type preservation (**sorry-free**; derives from evaluation equivalence via transitivity).

**Progress.** A synthesis-accepted term doesn't get stuck during evaluation. The evaluator may return a value or run out of fuel but never reaches an error state (**sorry-free**).

**End-to-end.** Composing the above:

$$\text{synth } e = \text{ok } a \wedge \text{subCheck } a \ b = \text{ok} \wedge e \Downarrow e' \Rightarrow \emptyset; \emptyset \vdash e' \sqsubseteq b$$

### 8.1 What soundness promises to the programmer

Two separable runtime properties a type system can promise:

1. **Termination.** Running a well-typed program eventually produces a value. A totality / normalization claim.
2. **No runtime type errors.** Conditional on the program *not* diverging, the value it produces is well-formed and the program never reaches a stuck state.

Och **does not aim to prove (1)**. Consistency and normalization are explicitly deferred; the calculus admits non-terminating terms by design (type-in-type; general recursion via fix).

Och **does aim to prove (2)**, and this is the real runtime guarantee of the type system. The language expects the following discipline: **run synthesis first; only evaluate if it succeeded**.

## 9 Appendix A. Declarative typing (derived)

Och has no declarative typing relation as a first-class definition; the algorithmic synthesis walk (§7) is the typing spec, with soundness against  $\sqsubseteq$  in §8. For readers who want the textbook form, the following relation  $\Gamma \vdash e : \tau$  is the clean projection of the algorithm.

$$\begin{array}{c}
\frac{}{\Gamma \vdash x : \Gamma(x)} \text{D-VAR} \qquad \frac{}{\Gamma \vdash \top : \top} \text{D-TYPE} \\
\\
\frac{\Gamma \vdash A : \top \quad \Gamma, x \leq A \vdash b : B}{\Gamma \vdash \lambda(x \leq A).b : \lambda(x \leq A).B} \text{D-LAM} \qquad \frac{\Gamma \vdash f : \lambda(x \leq A).B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B[x \mapsto a]} \text{D-APP} \\
\\
\frac{\Gamma \vdash v : A \quad \Gamma, x \leq A \vdash b : \tau}{\Gamma \vdash \text{let } x = v \text{ in } b : \tau} \text{D-LET} \qquad \frac{\Gamma \vdash \tau : \top \quad \Gamma \vdash e : \tau}{\Gamma \vdash (e \leq \tau) : \tau} \text{D-ASC} \\
\\
\frac{\Gamma \vdash A : \top \quad \Gamma, x \leq A \vdash B : \top \quad \Gamma \vdash v : A \quad \Gamma \vdash v : B[x \mapsto v]}{\Gamma \vdash v : \iota(x \leq A).B} \text{D-IOTA} \\
\\
\frac{\Gamma \vdash A : \top \quad \Gamma, x \leq A \vdash b : A}{\Gamma \vdash \text{fix}(x \leq A).b : A} \text{D-FIX} \qquad \frac{\Gamma \vdash e : A \quad \emptyset; \Gamma \vdash A \sqsubseteq B}{\Gamma \vdash e : B} \text{D-SUB}
\end{array}$$

## 10 Appendix B. Lean formalisation

The entire calculus described in this paper is formalised in Lean 4. This appendix maps the paper's named-variable presentation to the mechanised definitions, and notes the one systematic difference: the formalisation uses **de Bruijn indices** throughout, so named variables  $x$  become positional indices, substitution  $\text{body}[x \mapsto v]$  becomes index arithmetic, and context lookup  $\Gamma(x)$  becomes list indexing.

### 10.1 Representation

All terms, types, and values share a single inductive `Expr`. Named variables  $x$  are represented by `Expr.bvar` (a natural-number index); substitution uses `Expr.subst/Expr.shift` for standard de Bruijn arithmetic. The context  $\Gamma$  is `List Expr`, indexed by position with the innermost binder at index 0.

A named variable  $x$  bound at the innermost binder becomes `bvar 0`; a variable bound  $k$  binders out becomes `bvar k`; substitution  $\text{body}[x \mapsto v]$  becomes `body.subst 0 v`.

## 10.2 Correspondence

| Paper concept                                                | Lean definition                    | File                                             |
|--------------------------------------------------------------|------------------------------------|--------------------------------------------------|
| Term syntax ( $e, \tau$ )                                    | <code>Expr</code>                  | <code>Syntax.lean</code>                         |
| Concrete evaluation ( $e \Downarrow v$ )                     | <code>concEval</code>              | <code>Eval.lean</code>                           |
| Declarative subtyping ( $S; \Gamma \vdash a \sqsubseteq b$ ) | <code>Subtype'</code>              | <code>Subtyping.lean</code>                      |
| WHNF evaluator ( $e \Rightarrow v$ )                         | <code>evalSubst</code>             | <code>EvalSubst.lean</code>                      |
| Algorithmic subtype check ( $\sqsubseteq_e$ )                | <code>subCheckSubst</code>         | <code>EvalSubst.lean</code>                      |
| Type synthesis ( $\Gamma \vdash e \rightsquigarrow v$ )      | <code>synthCore</code>             | <code>API.lean</code>                            |
| Public entry point                                           | <code>0ch.synth</code>             | <code>API.lean</code>                            |
| Evaluation equivalence                                       | <code>concEval_equiv</code>        | <code>Soundness/ConcEvalPreservation.lean</code> |
| Preservation                                                 | <code>concEval_preservation</code> | <code>Soundness.lean</code>                      |
| Progress                                                     | <code>synth_progress</code>        | <code>Soundness.lean</code>                      |
| End-to-end soundness                                         | <code>soundness</code>             | <code>Soundness.lean</code>                      |

Table 3: Paper-to-Lean correspondence

## 10.3 De Bruijn details

The formalisation’s seen-set  $S$  is `List (Nat × Expr × Expr)` — each entry is depth-tagged with  $|\Gamma|$  at which it was recorded, and the hypothesis rule shifts entries to the current depth on use. This bookkeeping is invisible in the named presentation: free variables carry their own names, so no shift is ever required at lookup time.

Level-variables (§5.2) are encoded as `bvar` indices in a reserved high range (`levelOffset = 100_000_000`). The helpers `substL/shiftL` are level-aware counterparts of `Expr.subst/Expr.shift` that leave level-vars untouched.

## 10.4 Proof status

Evaluation equivalence, preservation, and progress are **sorry-free**. Synthesis soundness and subtype-check soundness carry upstream sorries at engineering walls (closeAll commutation, app-arm of synthesis). The end-to-end composition is sorry-free in its own body.

## Bibliography

- [1] P. Fu and A. Stump, “Self Types for Dependently Typed Lambda Encodings,” in *Rewriting and Typed Lambda Calculi (RTA-TLCA)*, Springer, 2014.
- [2] M. Brandt and F. Henglein, “Coinductive Axiomatization of Recursive Type Equality and Subtyping,” *Fundamenta Informaticae*, vol. 33, no. 4, pp. 309–338, 1998.
- [3] D. S. Hutchins, “Pure Subtype Systems,” in *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 2010, pp. 287–298. doi: 10.1145/1706299.1706334.
- [4] V. Pasquale and Á. García-Pérez, “Towards the Type Safety of Pure Subtype Systems,” in *34th EACSL Annual Conference on Computer Science Logic (CSL)*, in Leibniz International Proceedings in Informatics (LIPIcs), vol. 363. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, pp. 37:1–37:16. doi: 10.4230/LIPIcs.CSL.2026.37.